



COVENANT UNIVERSITY
CANAAANLAND, KM 10, IDIROKO ROAD,
P.M.B 1023, OTA, OGUN STATE, NIGERIA.

FILE OF EXAMINATION:
COLLEGE:
DEPARTMENT:
SESSION: 2025/2026
COURSE CODE: COS221
COURSE TITLE: Computer Programming I (Java)

B.Sc. EXAMINATION
SCIENCE AND TECHNOLOGY
COMPUTER AND INFORMATION SCIENCES
SEMESTER: OMEGA
CREDIT UNIT: 3

Time Allowed: 3 hours

Instructions:

- Attempt one question from each Section.
- Use comments in Section B and Section C to distinguish what task or activity is carried out by a block of code.

SECTION A (20 marks)

Question One

- Define method overloading in Java, and state precisely the role that the method signature plays in enabling the compiler to resolve an overloaded call at compile time. 20 marks
- State three differences between checked exceptions and unchecked exceptions in Java [2 Marks]
- This code is supposed to: Loop through the array, and for each starting index i , find the first number (from i onward) that's a multiple of 4. Add (that number - 1) to sum, count it as a match, stop after 3 matches, then display the sum and match count. Find the errors if the correct output should be: [3 Marks]
Sum: 141.0 Matches: 3 [5 Marks]

```

1 import javax.swing.JOptionPane;
2 public class Snippet {
3     public static void main(String[] args) {
4         int[] arr = {24.0, 40.0, 66.0, 80.0, 98.0};
5         int sum = 0;
6         int matches = 0;
7         boolean stop = false;
8         for (int i = 0; i <= arr.length && !stop; i++) {
9             for (int j = i; j < arr.length; j++) {
10                 if (arr[j] % 4 == 0) {
11                     continue;
12                 }
13                 sum += arr[j] - j;

```

```

14         matches++;
15         if (matches == 3) {
16             stop = true;
17         }
18         break;
19     }
20 }
21 JOptionPane.showMessageDialog(null,
22     "Sum: " + sum);
23 JOptionPane.showMessageDialog(null,
24     "Matches: " + matches);
25 System.exit(0);

```

- An Armstrong number (also called a narcissistic number) is a positive integer that is equal to the sum of its own digits, each raised to the power of the total number of digits in the number. For example, 153 is an Armstrong number because $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$.
 - State the five outputs for this code (you can show your workings but make the answer bold)
 - Retain other things but change the For Loop to While loop and the While loops to For loops. [10 Marks]


```

public class ArmstrongChecker {
    public static void main(String[] args) {
        int[] numbers = {370, 371, 372, 407, 408};
        for (int idx = 0; idx < numbers.length; idx++) {
            int n = numbers[idx], digits = 0, temp = n, sum = 0;
            while (temp > 0) { digits++; temp /= 10; }
            temp = n;
            while (temp > 0) {
                int d = temp % 10, power = 1;
                for (int p = 0; p < digits; p++) power *= d;
                sum += power;
                temp /= 10;
            }
            System.out.println(n + " -> " + (sum == n ? "ARMSTRONG" : "NOT
ARMSTRONG"));
        }
    }
}

```

20 marks
[2 marks]
[3 marks]

Question Two

- Define encapsulation as an OOP principle
- State three differences between Method overloading and Method overriding
- This code is supposed to loop through an array of prices, skip anything under 20, apply a 10% discount to prices 20 and above, add the discounted price to total, count it, and stop once 4 items have been discounted. Then it displays the total and count. Find the errors if the correct output should be:
Total: 202.725 Count: 4 [5 marks]

```

1 import javax.swing.JOptionPane;
2 public class DiscountCalc {
3     public static void main(String[] args) {
4         int[] prices = {15.5, 42.0, 8.75, 60.0,
5             33.25, 90.0};
6         int total = 0;
7         int count = 0;
8         boolean done = false;
9         for (int i = 0; i <= prices.length &&
10             !done; i++) {
11             if (prices[i] > 20) {
12                 continue;
13             }
14         }
15     }
16 }

```

```

12 double discount = prices[i] * 0.1;
13 total += prices[i] - discount;
14 count++;
15 if (count == 5) {
16     done = true;
17 }
18 }
19 JOptionPane.showMessageDialog(null,
20     "Total: " + total);
21 JOptionPane.showMessageDialog(null,
22     "Count: " + count);
23 System.exit(0);
24 }

```

- A perfect number is a positive integer that is equal to the sum of its own proper positive divisors (all divisors excluding the number itself). For example, 6 is perfect because $1 + 2 + 3 = 6$. A number whose proper divisors sum to more than the number itself is called abundant, and one whose proper divisors sum to less than the number itself is called deficient.

- State the output for this code (no need to show workings)
- Change the For loops to While loops

[10 marks]


```

public class NumberClassifier {
    public static void main(String[] args) {
        int perfectCount = 0, abundantCount = 0, deficientCount = 0;
        for (int num = 1; num <= 30; num++) {
            int sum = 0;
            for (int div = 1; div < num; div++) {
                if (num % div == 0) sum += div;
            }
            if (sum == num) perfectCount++;
            else if (sum > num) abundantCount++;
            else deficientCount++;
        }
        System.out.println("Perfect: " + perfectCount + ", Abundant: " +
            abundantCount + ", Deficient: " + deficientCount);
    }
}

```

SECTION B (25 marks)

Question Three

A. A store records daily sales in an integer array `saleAmounts[]`, in the order the sales happened. A "consecutive duplicate" is any position i ($i \geq 1$) where `saleAmounts[i] == saleAmounts[i-1]`. (12 marks)

Given the array:

Index	0	1	2	3	4	5	6	7	8	9
Amount	20	20	35	40	40	40	15	60	60	25

Write a program that:

- Create a method that makes **one pass** through the array and returns the count of indices $i \geq 1$ where `saleAmounts[i] == saleAmounts[i-1]`.
- Create a method that returns a single String listing, in original order and comma-separated, every sale amount that equals its predecessor or its successor in the array. Build this string using plain String concatenation only (no StringBuilder, no array of booleans returned). Index 0 checks only its successor; the last index checks only its predecessor.
- Using `JOptionPane`, display one dialog showing (a) the total consecutive-duplicate count from part 1, and (b) the flagged-amounts string from part 2.

[Hint: To aid your flow trace your algorithm by hand first: it must not flag 35, 15, or 25.]

B. A college admin office needs a program to record and review student details (ID, name, CGPA) using basic file I/O. Using **only** `java.io.FileWriter`, `java.io.FileReader`, `java.io.BufferedReader`, `java.io.BufferedWriter`, and `java.io.IOException` (or its subclasses), write a complete program that does the following: (13 marks)

- Create a file named `input.txt` and write the details (student ID, name, CGPA) of at least three students into it using `FileWriter`, one student per line. Wrap the writing logic in a try-catch block that handles any exception that may occur.
- Read student details from a file named `output.txt` using `FileReader` and print each line to the console exactly as it appears in the file. Handle any exception that may occur while reading, including the case where `output.txt` does not exist.
- Create a method that takes in a file name, opens the file, counts the number of lines (student records) it contains, and returns that count. This method must handle its own exceptions internally.

- iv. Ensure every file resource opened is properly closed (either in a finally block or via try-with-resources) and that every catch block prints a clear, specific message describing what went wrong.

Console output (System.out) is acceptable throughout.

25 marks

Question Four

A bank wants a system representing different types of accounts using inheritance and polymorphism.

(9 marks)

Part 1: Superclass BankAccount

- Create a class named BankAccount with **private** fields to hold the account number, the account holder's name, and the balance. Provide a constructor to initialise all three.
- Create a method that adds a given amount to the balance.
- Create a method that deducts a given amount from the balance if there are sufficient funds; otherwise, the transaction should be rejected and a message displayed.
- Create a method that displays the account's current details: its account number, holder's name, and balance.

(10 marks)

Part 2: Subclasses

- SavingAccount extends BankAccount. Override the withdrawal method so a withdrawal is only allowed if the balance **after** the withdrawal would still be $\geq \text{₦}1,000$. Otherwise, reject and display a message.
- CurrentAccount extends BankAccount. Override the withdrawal method so that every **successful** withdrawal automatically adds a $\text{₦}50$ reward to the balance afterward.

(6 marks)

Part 3: Main Method

- Creates at least one SavingAccount and one CurrentAccount;
- Deposits money into each;
- Performs at least one successful **and** one unsuccessful/reward-triggering withdrawal on each;
- Displays each account's details before and after the transactions;
- Demonstrates polymorphism by storing each object in a BankAccount reference and calling the withdrawal method through it, confirming that the correct overridden version runs.

SECTION C (25 marks)

25 marks

Question Five

A. A conference organiser needs a program (using JOptionPane only, for all input/output) that prices tickets for a 3-day conference and produces a sales summary. Implement the following four methods.

(12 marks)

- Create a method that takes in a day number. Use a switch statement to return a base ticket price for Day 1, Day 2, or Day 3 (pick any three distinct prices and state them clearly). If the day number is not 1, 2, or 3, throw an IllegalArgumentException.
- Create a method that takes in a tier and a number of days attending. Use **nested if-else** statements (not a switch, and not independent/separate if checks) to return a discount percentage:

- Gold, attending all 3 days $\rightarrow 25\%$
- Gold, attending 1-2 days $\rightarrow 15\%$
- Silver, attending all 3 days $\rightarrow 15\%$
- Silver, attending 1-2 days $\rightarrow 5\%$
- Bronze (any days) $\rightarrow 0\%$
- Any tier string that is not "Gold" or "Silver" must be treated as "Bronze"

The nesting must reflect that "days attending" is only checked within each tier branch, not as a separate top-level check.

- iii. Create a method that takes in a base price total and a discount percentage and applies the discount to the base price total. If the discounted amount is below 50.0, the method should return 50.0 instead (a price floor).
- iv. `main()`
 - Ask the user for the number of attendees N (a positive integer).
 - Loop exactly N times. In each iteration, ask for that attendee's number of days attending (an integer from 1 to 3, inclusive) and tier. For each day from 1 up to that attendee's days attending, call the method from part (i) and add the result to that attendee's running base price total (a running total that starts at 0 for each new attendee). Then call the method from part (ii) to get the discount percentage, and the method from part (iii) to get that attendee's final price.
 - Accumulate: (a) total revenue across all attendees, and (b) a running count of how many attendees were Gold, Silver, and Bronze.
- v. After the loop, show one summary dialog with total revenue and the three tier counts.

B. A cold-storage warehouse records its internal temperature once every hour, for 24 hours. Write a program, using `JOptionPane` only for input/output, that checks the day's temperature log for sustained breaches of a safe limit. A reading is a **breach** if it is **above 8°C** . (13 marks)

- i. Using a loop, prompt the user 24 times (one dialog per hour) for that hour's temperature. Each input must be validated: it must parse as a number between **-20 and 50 inclusive**. If invalid, re-prompt for the *same* hour (using a nested loop/control statement) until a valid value is entered. Store all 24 valid readings in a 1-D array.
- ii. Scan the 24-hour array once to find every **maximal run of consecutive breach-hours** (temperature $> 8^{\circ}\text{C}$). Using only loops and conditionals (no extra arrays), determine:
 - the total number of qualifying streaks, where a qualifying streak is any consecutive run of 3 or more breach-hours;
 - the length (in hours) of the single longest breach streak anywhere in the log.
- iii. Using nested if-else logic, classify the day as exactly one of:
 - "CRITICAL": longest streak ≥ 6 hours
 - "WARNING": longest streak is 3-5 hours (inclusive)
 - "SAFE": no streak of 3 or more consecutive breach-hours occurred
- iv. Show one `JOptionPane` dialog stating: the total number of qualifying streaks, the longest streak length, and the final classification.

25 marks

Question Six

A. Design and implement a program, using `JOptionPane` only for input and output, that classifies a single user-supplied positive integer N according to its digit pattern (Palindrome).

Definition: A number is a **palindrome** if it reads the same forwards and backwards (i.e. it is equal to its own digit-reversed value). For example, 4884 and 7 are palindromes; 4821 is not.

Constraints: The entire task must be implemented inside `main()` only, using loops and nested/chained if-else and/or switch logic. No arrays and no user-defined helper methods are permitted; everything must be inline. (12 marks)

- i. Using a loop built only on the `%` and `/` operators (no converting N to a String or array), compute, in the same loop:

- the **digit sum** of N , and
 - the value of N with its **digits reversed** (e.g. for $N = 4821$, the reversed value is 1284).
- Determine whether N is a palindrome by comparing N directly to the reversed value computed in step 1.
 - Inline in `main()` (not a separate method), determine whether the **digit sum** from step 1 is itself a prime number. Use a loop that tests divisibility only up to the **square root** of the digit sum.
 - Using nested/chained conditional logic, classify N into **exactly one** of the following categories, checked strictly in this priority order:
 - **"PALINDROME-PRIME-SUM"**: if N is a palindrome **AND** its digit sum is prime
 - **"PALINDROME"**: if N is a palindrome but its digit sum is **not** prime
 - **"HARSHAD"**: if N is **not** a palindrome but N is exactly divisible by its own digit sum
 - **"STANDARD"**: in every other case
 - Display N , its digit sum, its reversed value, and its final classification in a **single JOptionPane** message dialog.

B. A university parking garage needs a program, using **JOptionPane** only for input/output, to process a batch of vehicles exiting on the same day and calculate the total fees owed. The program must use clear separation of concerns; each task below must be its own method. No collection classes (e.g. ArrayList) are allowed anywhere. **(13 marks)**

- Prompt for the number of exiting vehicles, v . Then, using a loop, fill two **parallel arrays** of length v :
 - the number of hours each vehicle was parked (an integer per vehicle). A value ≤ 0 is invalid; use a **nested control statement** to keep re-prompting until a valid value is entered.
 - the permit type for each vehicle (a String per vehicle, either "Staff" or "Visitor").
- Create a method that takes in one vehicle's hours parked and permit type, and computes one vehicle's fee using **nested if-else logic** (the rules below must be nested together, not checked as separate independent conditions):
 - **Staff**: first 2 hours free; each additional hour costs 1.00; capped at a maximum of 10.00/day.
 - **Visitor**: no free period; first 3 hours cost a flat 6.00; each hour beyond 3 costs 2.50; capped at a maximum of 25.00/day.
 - Any permit type that is not exactly "Staff" must be treated as "Visitor".
- Create a method that takes in the array of hours parked, loops over the array and tallies vehicles into three categories: **SHORT-STAY** (1–2 hrs), **STANDARD** (3–6 hrs), **EXTENDED** (>6 hrs). Return these three counts without using any collection class.
- In `main()`,
 - call the method from part (ii) for every vehicle across both parallel arrays to accumulate the batch's total revenue
 - call the method from part (iii) once
 - display one **final JOptionPane** showing the total revenue and the three category counts.

COVENANT UNIVERSITY OTA
COMPUTER & INFO. SCIENCES

HEAD OF DEPARTMENT

Date: 03/07/26